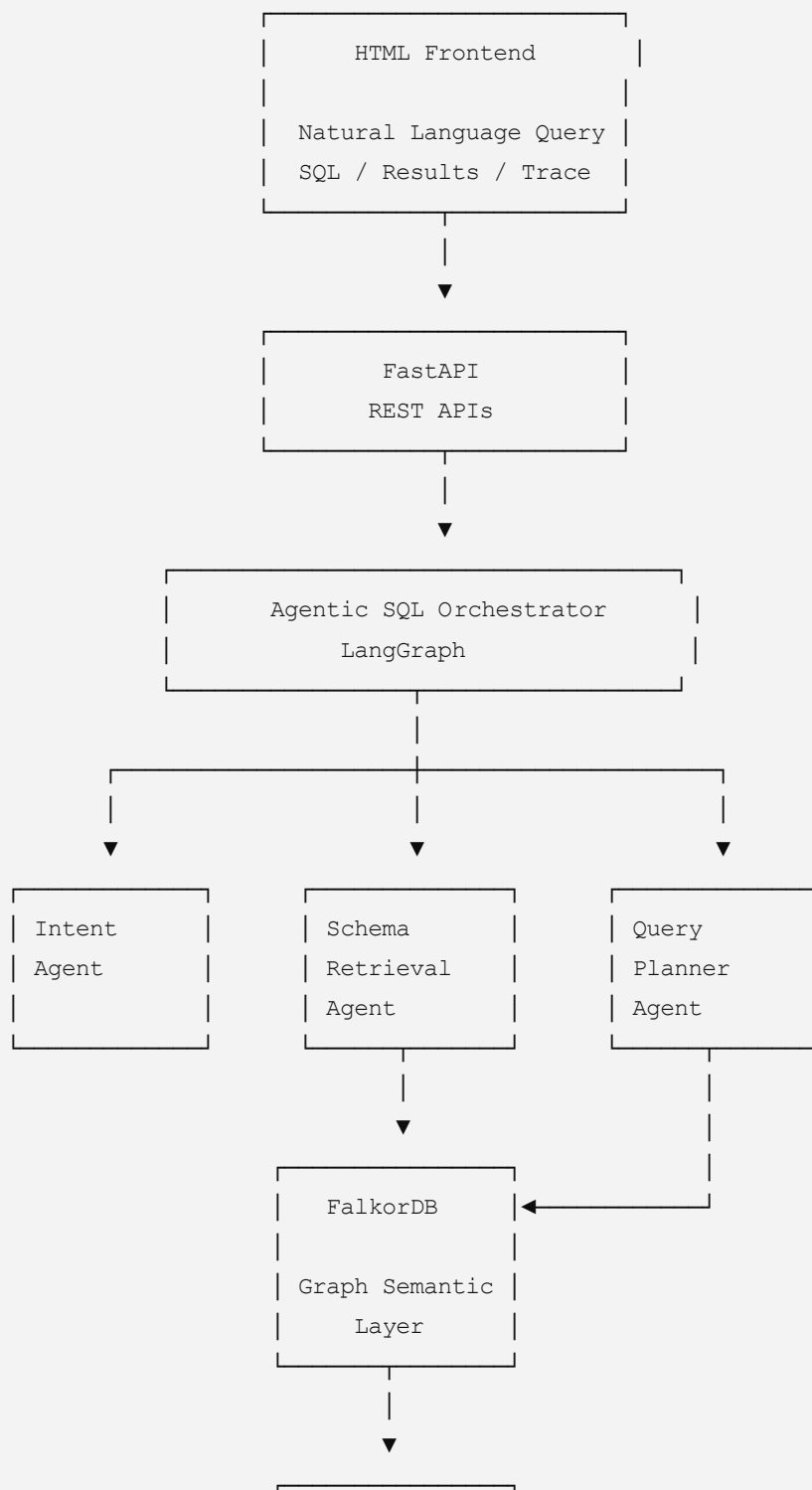


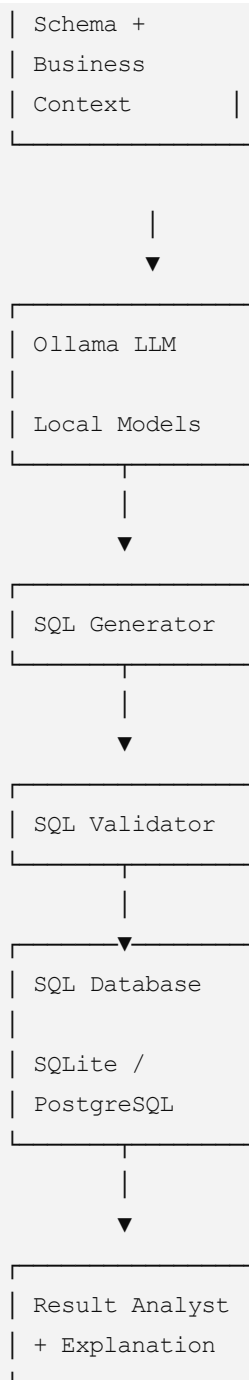
Absolutely. This is a **very good learning project**, and I would recommend building it without trying to make it production-grade initially. We can progressively evolve it from a simple Text-to-SQL application into a proper **Agentic AI Text-to-SQL platform**.

One small clarification: I assume by “**FalcorDB**” you mean **FalkorDB**. FalkorDB is a graph database that supports Cypher and is designed for low-latency graph traversal; it can run locally with Docker.  FalkorDB Kno...

1. Target architecture

I suggest we eventually build this architecture:





The important idea is that **FalkorDB is not the database being queried by the generated SQL.**

Instead:

Business data → SQL database

Business/schema relationships → FalkorDB semantic graph

Natural language → agents → semantic graph → SQL → database

That separation will make the project particularly useful for learning AI architecture.

2. Technology stack

I'd recommend this stack for the first version.

Layer	Technology
Frontend	HTML + CSS + JavaScript
Backend	FastAPI
Agent orchestration	LangGraph
LLM	Ollama
Initial LLM	Qwen / Llama / Gemma family
Graph semantic layer	FalkorDB
Graph language	Cypher
SQL database	SQLite initially
ORM / DB abstraction	SQLAlchemy
Embeddings	Ollama embeddings
Vector search	Initially optional
Configuration	<code>.env</code>
Logging	Python logging
Testing	pytest
Containerization	Docker later

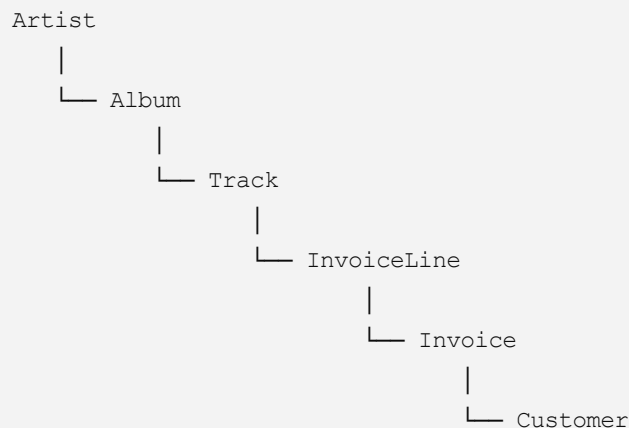
Ollama is particularly suitable here because it allows us to run models locally rather than depending on a paid API.  Ollama

3. Which database should you use?

I strongly recommend **Chinook SQLite** for Phase 1.

Chinook represents a digital media store and contains entities such as artists, albums, tracks, customers, invoices and employees. It is available for SQLite, PostgreSQL, MySQL, SQL Server and several other databases.  GitHub +1

This is actually excellent for Text-to-SQL because it has relationships such as:



This gives us enough complexity to demonstrate graph reasoning.

Other free databases

Database	Best use
Chinook	★ Best starting point
Northwind	★ Excellent second database
AdventureWorks	Larger SQL learning
Pagila	PostgreSQL
Sakila	MySQL
NYC Taxi	Large-scale analytics
Netflix-style datasets	Analytics / aggregation

Northwind is another good choice because it contains customers, products, suppliers, employees, orders and order details. [GitHub](#)

There is also a Dockerized PostgreSQL Northwind database if we later want to move from SQLite to PostgreSQL. [GitHub](#)

4. Recommended learning progression

Rather than building everything at once, let's build **10 phases**.

Phase 0 — Environment

Set up:

```
Python
Ollama
Docker
FalkorDB
SQLite
VS Code
Git
```

Phase 1 — Simple Text-to-SQL

Build:

```
User question
      ↓
Ollama
      ↓
SQL
      ↓
SQLite
      ↓
Result
```

Example:

Show the top 5 artists by number of tracks.

LLM produces:

```
SQL

SELECT
    a.Name,
    COUNT(t.TrackId) AS track_count
FROM Artist a
JOIN Album al
    ON a.ArtistId = al.ArtistId
JOIN Track t
    ON al.AlbumId = t.AlbumId
GROUP BY a.ArtistId, a.Name
ORDER BY track_count DESC
LIMIT 5;
```

This gives us our baseline.

5. Phase 2 — Schema-aware Text-to-SQL

Instead of blindly asking Ollama:

Question → LLM

we'll give it database metadata:

```
Question
+
Database schema
↓
Ollama
↓
SQL
```

For example:

```
TABLE Artist
- ArtistId INTEGER PRIMARY KEY
- Name VARCHAR

TABLE Album
- AlbumId INTEGER PRIMARY KEY
- Title VARCHAR
- ArtistId INTEGER FOREIGN KEY → Artist.ArtistId

TABLE Track
- TrackId INTEGER PRIMARY KEY
- Name VARCHAR
- AlbumId INTEGER FOREIGN KEY → Album.AlbumId
```

This dramatically improves SQL generation.

6. Phase 3 — Introduce FalkorDB

This is where your project becomes interesting.

We will create a graph such as:

```
(:Table {name:"Artist"})
|
| HAS_COLUMN
↓
(:Column {name:"ArtistId"})
```

and:

```
(:Table {name:"Album"})
  |
  | HAS_COLUMN
  ↓
(:Column {name:"ArtistId"})
  |
  | REFERENCES
  ↓
(:Column {name:"Artist.ArtistId"})
```

More conceptually:

```
Artist
  |
  | 1:N
  ▼
Album
  |
  | 1:N
  ▼
Track
  |
  | 1:N
  ▼
InvoiceLine
  |
  ▼
Invoice
  |
  ▼
Customer
```

Now we can ask FalkorDB:

What tables are relevant to answering this question?

and retrieve something like:

```
Artist
Album
Track
InvoiceLine
Invoice
Customer
```

The LLM doesn't need to figure out the entire database topology itself.

7. Phase 4 — Graph Semantic Layer

Then we'll enrich the graph.

Instead of storing only:

```
Table → Column
```

we'll store:

```
Table
Column
Relationship
Business Concept
Metric
Synonym
Description
Join Path
```

For example:

```
Customer
|
├─ customer_id
├─ country
├─ first_name
└─ last_name
```

Business Concept:

Customer

Synonyms:

client

buyer

consumer

account

And:

```
Revenue
|
├─ Invoice
└─ InvoiceLine
```

with a semantic definition:

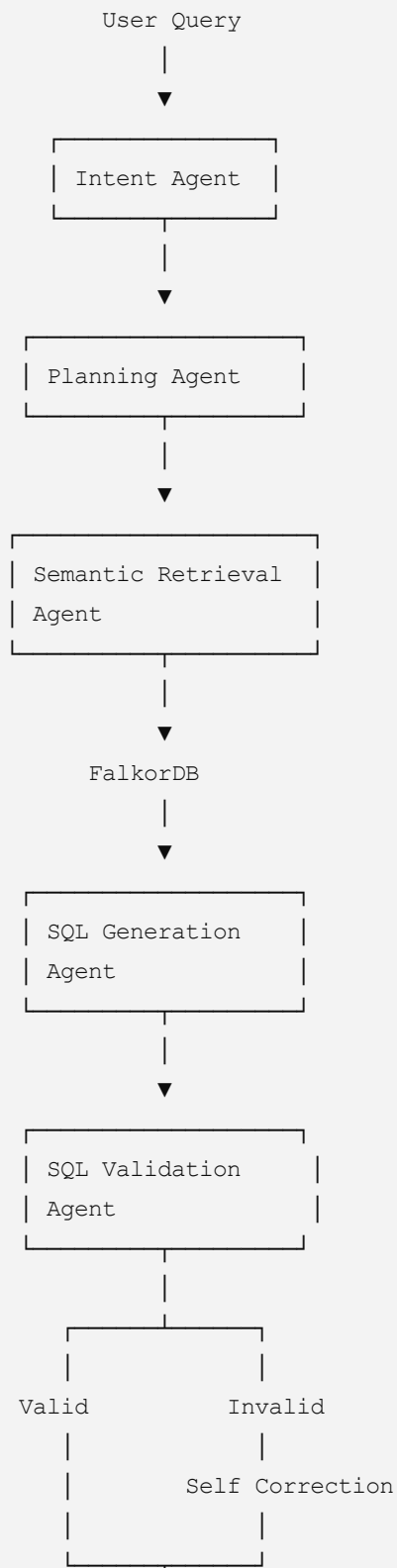
```
Revenue =
```

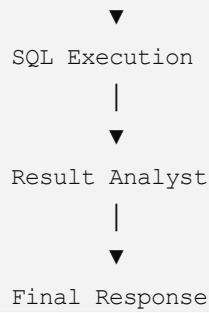

This is where the graph becomes a **semantic layer**, rather than merely a schema catalog.

8. Phase 5 — Agentic architecture

Then we introduce actual agents.

I'd suggest:





This will give you hands-on experience with **agent orchestration** rather than simply **calling an LLM**.

9. Phase 6 — Self-correction

Suppose Ollama generates:

```
SQL

SELECT
  a.Name,
  COUNT(t.TrackID)
FROM Artist a
JOIN Track t
  ON a.ArtistID = t.ArtistID
GROUP BY a.Name;
```

but `Track` doesn't contain `ArtistID`.

The execution layer returns:

```
SQL Error:
no such column: t.ArtistID
```

Instead of returning the error:

```
SQL Generator
  ↓
SQL Validator
  ↓
Execution
  ↓
ERROR
  ↓
Correction Agent
  ↓
FalkorDB
  ↓
Correct JOIN path
```

↓
New SQL

That is a much more meaningful **agentic workflow**.

10. Phase 7 — Explainability

The UI should eventually show:

Question

Who are the top 5 artists by sales?

Intent

Sales analysis

Semantic concepts

Artist

Track

InvoiceLine

Invoice

Join path

Artist

↓

Album

↓

Track

↓

InvoiceLine

↓

Invoice

Generated SQL

SELECT ...

Validation

✓ Syntax valid

✓ Tables valid

✓ Columns valid

✓ Join path valid

Execution

✓ Successful

Confidence

92%

Answer

The top 5 artists are...

This will be very useful when you later want to demonstrate the architecture in interviews or architectural discussions.

11. Phase 8 — Memory

Later we'll add:

Short-term memory

Current conversation:

User:

Show sales by country.

Agent:

...

User:

Only for 2025.

Agent:

...

The second query understands that:

"Only for 2025"

refers to the previous query.

Long-term memory

We could remember:

User frequently asks:

- Sales
- Customers
- Revenue

or successful SQL patterns.

12. Phase 9 — Evaluation

We'll eventually create a benchmark:

Question	Expected SQL
-----	-----
Top 5 artists	SQL-001
Revenue by country	SQL-002
Best selling album	SQL-003
Customers from Brazil	SQL-004
Monthly revenue	SQL-005

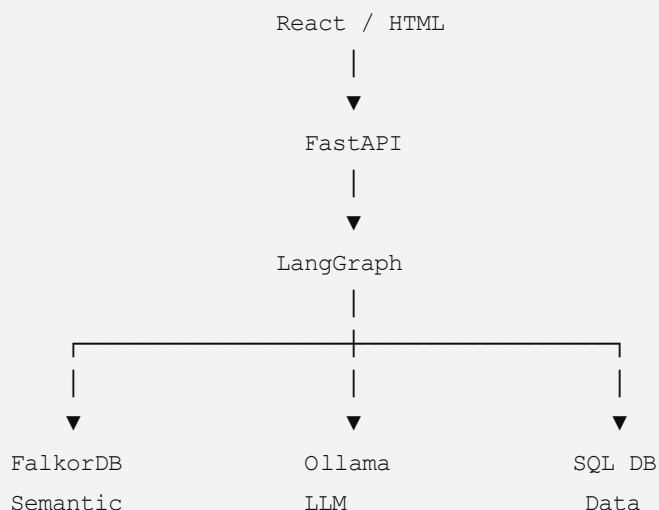
Then measure:

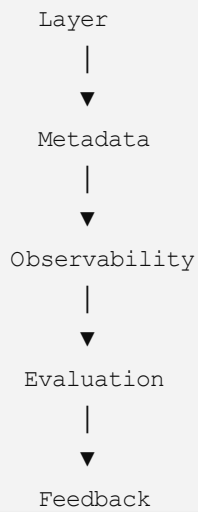
SQL execution accuracy
SQL semantic accuracy
Schema selection accuracy
Join-path accuracy
Answer accuracy
Correction rate
Latency
Token usage
Confidence calibration

This is particularly important for a real Agentic AI architecture.

13. Phase 10 — Production-style architecture

Finally:





At that point we can add:

- authentication
- conversation history
- SQL editor
- graph visualization
- agent trace
- evaluation dashboard
- feedback mechanism
- human-in-the-loop
- query caching
- prompt management
- model selection
- confidence scoring
- audit logging
- Docker Compose
- deployment

14. Initial setup

Since you're on Windows, I would use:

```
Windows
|
├── VS Code
├── Python 3.11/3.12
├── Git
├── Docker Desktop
|
├── Ollama
│   └── Local LLM
|
```

```
└─ Project
   └─ FastAPI
   └─ LangGraph
   └─ SQLAlchemy
   └─ FalkorDB
```

Step 1 — Install Ollama

Install Ollama from its official site. It supports running open models locally.  Ollama

Ollama official website

After installation:

Bash

```
ollama --version
```

Then pull a model.

For this project, I'd start with a relatively capable coding/reasoning model rather than the smallest model.

For example, depending on your machine:

Bash

```
ollama pull qwen3:8b
```

or a larger model if your machine has sufficient RAM/VRAM.

Then:

Bash

```
ollama run qwen3:8b
```

Test:

```
Write a SQL query to find the top 5 customers by revenue.
```

15. Step 2 — Install Docker Desktop

We'll use Docker for FalkorDB.

After Docker is installed:

Bash

```
docker --version
```

Then:

Bash

```
docker run -p 6379:6379 -it --rm falkordb/falkordb:latest
```

FalkorDB documents this Docker-based local setup and provides a Python client that can be installed with `pip install falkordb`.  FalkorDB Kno...

Later, instead of the raw Docker command, we'll create:

```
docker-compose.yml
```

with:

```
FastAPI
FalkorDB
```

and eventually possibly:

```
Ollama
```

16. Step 3 — Create the project

I'd structure it from the beginning like this:

```
agentic-text2sql/
|
├─ app/
|   └─ main.py
|   |
|   └─ api/
|       ├── routes.py
|       └─ schemas.py
|   |
|   └─ agents/
|       ├── planner.py
|       ├── schema_agent.py
|       └─ sql_generator.py
```



```
| | | └─ sql_validator.py
| | | └─ executor.py
| | | └─ analyst.py
| |
| | └─ graph/
| | | └─ client.py
| | | └─ schema_loader.py
| | | └─ semantic_search.py
| | | └─ queries.py
| |
| | └─ database/
| | | └─ connection.py
| | | └─ metadata.py
| | | └─ executor.py
| |
| | └─ llm/
| | | └─ ollama.py
| | | └─ prompts.py
| |
| | └─ workflow/
| | | └─ state.py
| | | └─ graph.py
| |
| | └─ utils/
| | | └─ logger.py
| | | └─ config.py
| |
| └─ frontend/
| | └─ index.html
| | └─ app.js
| | └─ style.css
| |
| └─ data/
| | └─ chinook.db
| |
| └─ tests/
|
| └─ .env
| └─ .gitignore
| └─ requirements.txt
| └─ Dockerfile
| └─ docker-compose.yml
| └─ README.md
```

I recommend **not** starting with all these modules implemented. We'll create the structure progressively.

17. Step 4 — Python environment

Create:

Bash

```
mkdir agentic-text2sql
cd agentic-text2sql

python -m venv .venv
```

Activate on Windows:

Bash

```
.venv\Scripts\activate
```

Then:

Bash

```
python -m pip install --upgrade pip
```

Initial packages:

Bash

```
pip install fastapi
pip install uvicorn
pip install sqlalchemy
pip install langchain
pip install langgraph
pip install ollama
pip install falkordb
pip install python-dotenv
pip install pydantic
```

For development:

Bash

```
pip install pytest
```

18. Step 5 — Download Chinook

For our first version, use the SQLite database.

The Chinook project provides a ready-made SQLite database file, so you don't need to create the schema manually.  [GitHub](#) +1

Put it here:

```
agentic-text2sql/  
└─ data/  
    └─ chinook.db
```

We can then connect using:

```
sqlite:///data/chinook.db
```

19. Step 6 — Test the database

Before introducing AI, make sure:

```
Python  
  ↓  
SQLite  
  ↓  
SQL  
  ↓  
Results
```

works.

For example:

SQL

```
SELECT  
    Name  
FROM Artist  
LIMIT 10;
```

Then:

SQL

```
SELECT  
    a.Name,  
    COUNT(t.TrackId) AS tracks  
FROM Artist a  
JOIN Album al
```

```
    ON a.ArtistId = al.ArtistId
JOIN Track t
    ON al.AlbumId = t.AlbumId
GROUP BY a.ArtistId, a.Name
ORDER BY tracks DESC
LIMIT 5;
```

This gives us our baseline before adding agents.

20. Step 7 — Test FalkorDB

Start:

Bash

```
docker run -p 6379:6379 -it --rm falkordb/falkordb:latest
```

Then Python:

Python

Run

```
from falkordb import FalkorDB

db = FalkorDB(
    host="localhost",
    port=6379
)

graph = db.select_graph("text2sql")

graph.query("""
CREATE (:Table {
    name: 'Artist',
    description: 'Contains music artists'
})
""")
```

Then:

Python

Run

```
result = graph.query("""
MATCH (t:Table)
RETURN t.name, t.description
""")

for record in result.result_set:
    print(record)
```

If you see:

```
Artist
Contains music artists
```

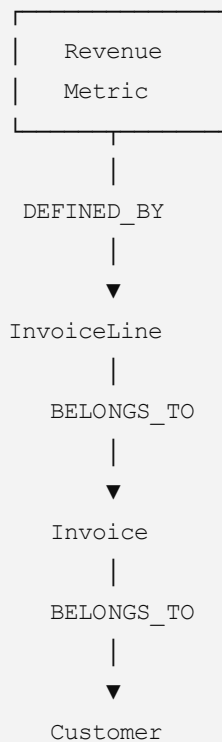
your graph layer is working.

21. The most important design decision

I would **not** initially use FalkorDB as just a replacement for a vector database.

We should make it a genuine **Graph Semantic Layer**.

For example:



And:

```
Revenue
|
├─ synonyms → sales
├─ synonyms → income
├─ synonyms → turnover
├─
└─ definition
    SUM(UnitPrice * Quantity)
```

Then the agent can reason:

User said "sales", which maps to the `Revenue` business concept.

Then:

Revenue requires InvoiceLine.

Then:

Customer requires Invoice.

Then:

Join path is InvoiceLine → Invoice → Customer.

Then:

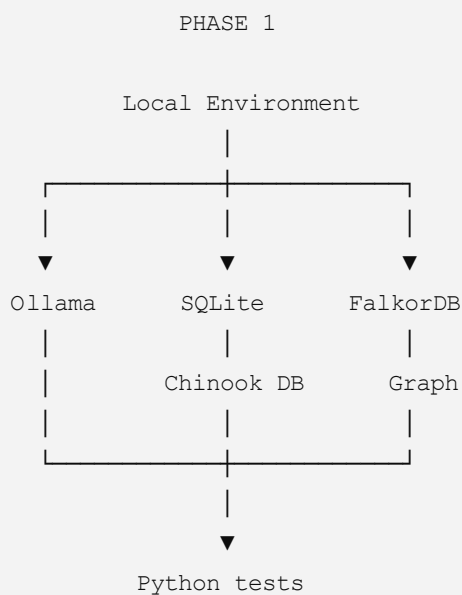
Generate SQL.

That's the part I'd really like us to build carefully.

22. Our first milestone

I suggest we **do not jump into LangGraph yet**.

Our first milestone should be:



At the end of Phase 1 we should be able to demonstrate:

- ✓ Ollama running
- ✓ Local LLM responding
- ✓ Chinook SQLite working

- ✓ Python querying SQLite
- ✓ FalkorDB running
- ✓ Python connecting to FalkorDB
- ✓ Basic graph created
- ✓ FastAPI running
- ✓ HTML frontend loading

Only after that should we start Phase 2 and build the first Text-to-SQL agent.

Recommended final evolution

The project will eventually evolve like this:

```
LEVEL 1
Simple Text → SQL
      ↓
LEVEL 2
Schema-aware Text → SQL
      ↓
LEVEL 3
Graph Semantic Layer
      ↓
LEVEL 4
LangGraph Agent
      ↓
LEVEL 5
SQL Validation + Self Correction
      ↓
LEVEL 6
Multi-Agent Architecture
      ↓
LEVEL 7
Memory + Human-in-the-loop
      ↓
LEVEL 8
Evaluation + Observability
      ↓
LEVEL 9
Explainability + Confidence
      ↓
LEVEL 10
Production-ready Agentic Text-to-SQL
```

This approach is also consistent with the kind of architecture you're interested in: **LLM + RAG/semantic context + graph reasoning + agents + tool calling + evaluation**, rather than merely building a chatbot that happens to generate SQL.

There are existing Text-to-SQL examples using Chinook and LangGraph, which is useful as a reference, but for your learning project I'd deliberately build the architecture ourselves rather than copying an existing implementation.  [GitHub](#)

Next step

I suggest we start with **Phase 1 hands-on**. I can walk you through it command-by-command on Windows: **install Ollama → select the right local model based on your RAM/GPU → install Docker → start FalkorDB → download Chinook → create the Python virtual environment → install dependencies → create the project structure → create a minimal FastAPI + HTML application → test all three components.**